# Performance TCP : fenêtre dynamique et minuterie

**GEF330 – Hiver 2020**
**CRI16**

Mariam El Mezouar
Collège Militaire Royal du Canada
Génie électrique et génie informatique
Mariam.el-mezouar@rmc-cmr.ca
elmezouar.segfaults.net

# Références

- Tanenbaum, § 6.2.2 – 6.2.3, 6.5.5 — 6.5.9
- Stevens, W.R., TCP/IP Illustrated Vol I, Addison-Wesley

# Aperçu

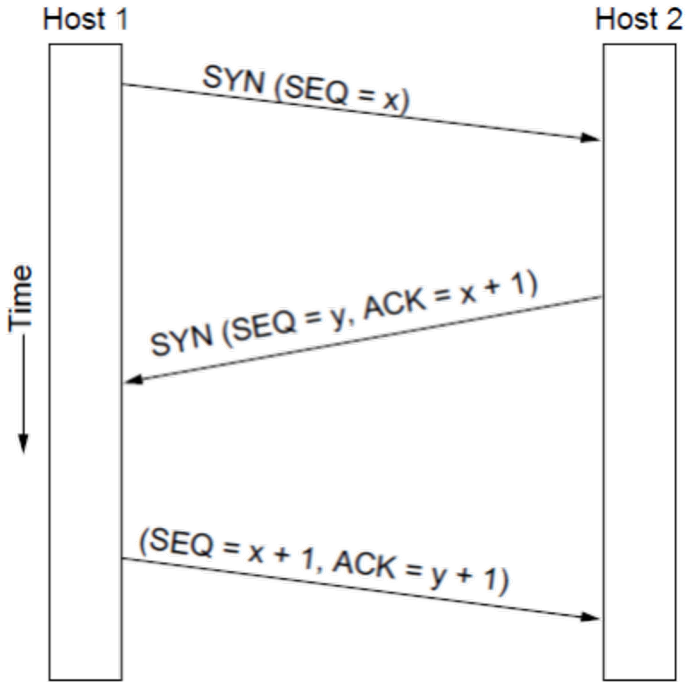
- Introduction sur la performance TCP.
- Révision de connexion TCP
- Fenêtre TCP
  - Fenêtre dynamique
  - Fenêtre de contrôle de flux
- Accusé réception en retard
  - Algorithme de Nagle
  - Algorithme de Clarke
- Minuterie TCP
  - Calcul de période (Jacobson)

# Introduction

- Jusqu'à présent, nous avons examiné comment TCP / IP peut transférer des données du point A au point B.
- Bien que l'encapsulation permette à nos différents protocoles de travailler ensemble pour déplacer des données, elle introduit également une certaine quantité de surcharge avec chaque élément de données que nous envoyons.
- Quel genre d'impact cela pourrait-il avoir pour:
  - Telnet
  - HTML
  - Chat textuel
  - Streaming
- Alors, comment pouvons-nous être plus efficaces?

# Révision : Établissement de connexion

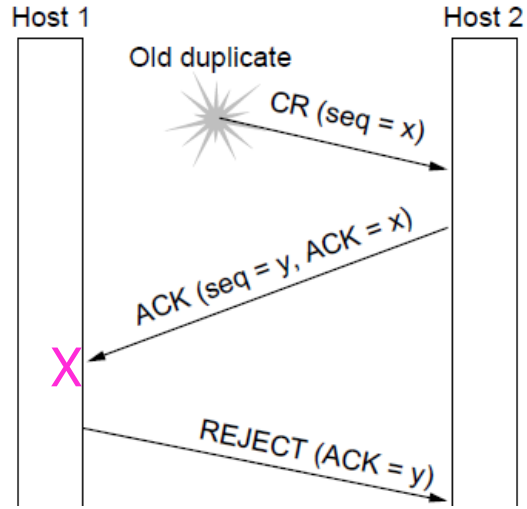
- Connexion en trois étapes
  - Puisque qu'il n'y a aucun état à partir de connexions précédentes, chaque hôte contribue de nouveaux nº de séquence/ACK
  - CR = Requête de connexion (Connect Request)



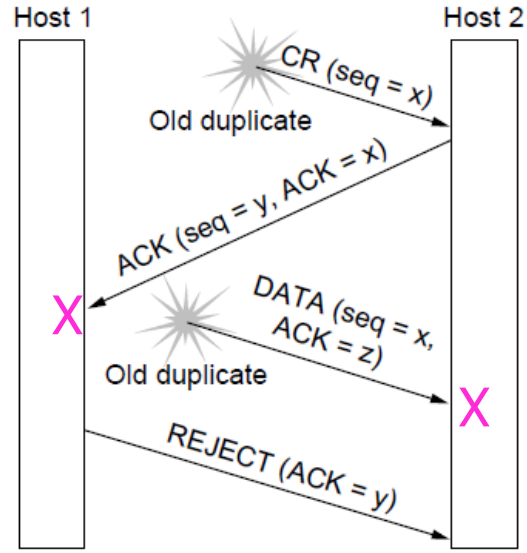
# Révision : Établissement de connexion

- Connexion en trois étapes protège contre les exceptions
  a. CR redondant, ACK parasites ne se connectent pas
  b. CR redondant et données ne se connectent pas, et les données sont rejetées (mauvais ACK)

a)



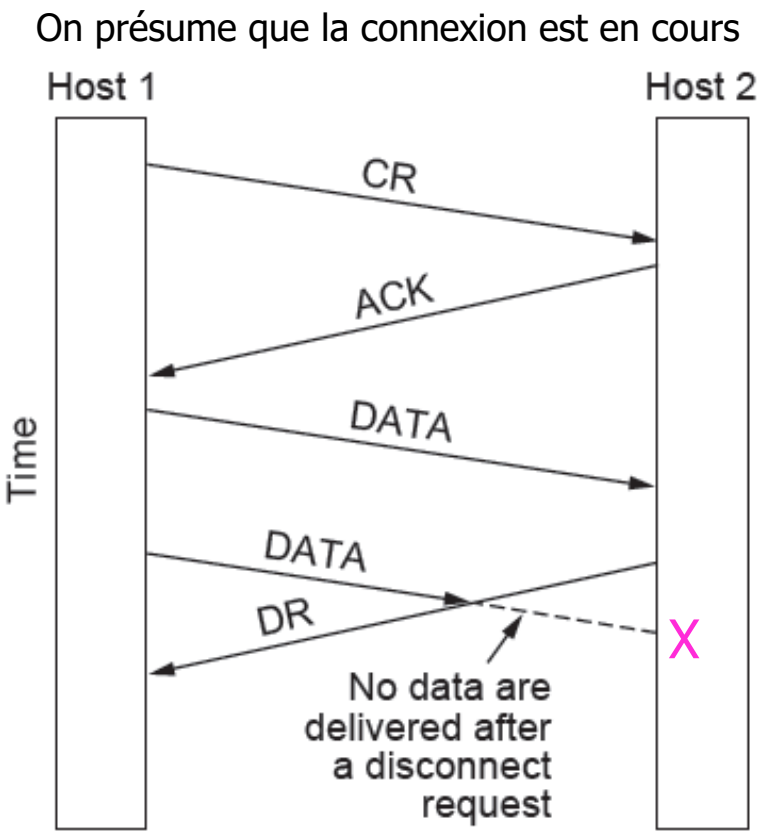

b)



# Révision : Relâchement de la connexion

On veut assurer la fiabilité lors du relâchement

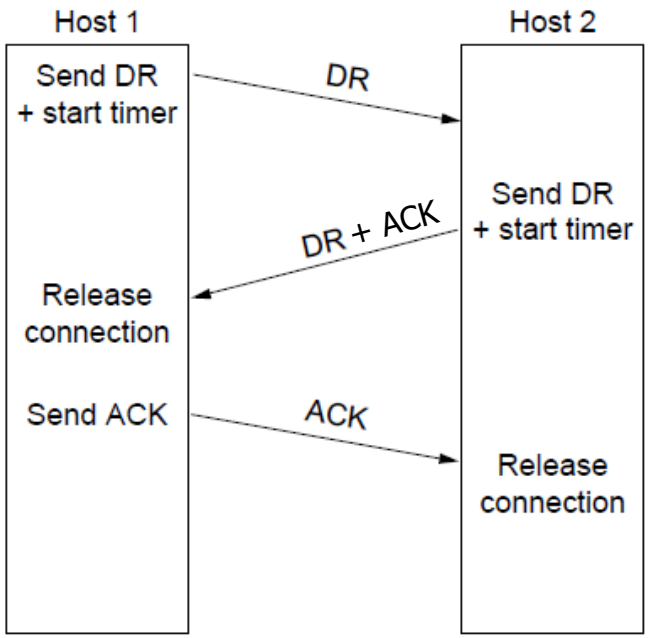
Les relâchements asymétriques (lorsqu'un côté rompt la connexion) sont brusques et risquent une perte des données




*DR = Disconnect Request*

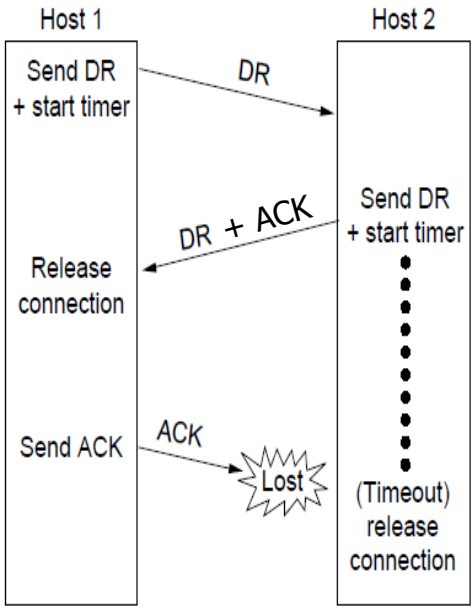
# Révision : Relâche de connexion

- Séquence de relâche habituelle
  - initiée par la couche transport sur Hôte 1
  - DR = requête de relâche
  - Les 2 DR reçoivent un ACK de l'autre côté

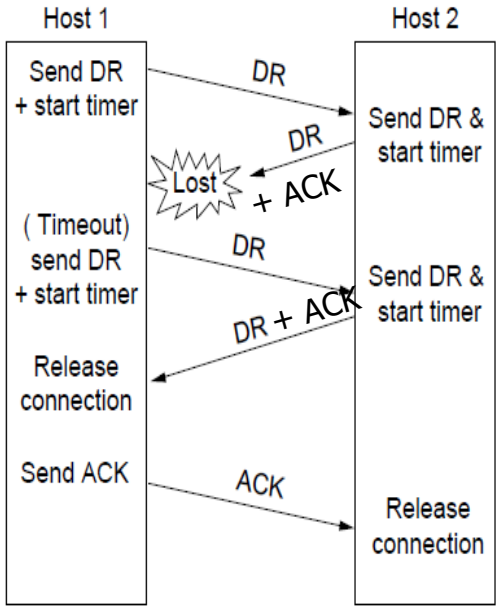


# Révision : Relâche de connexion

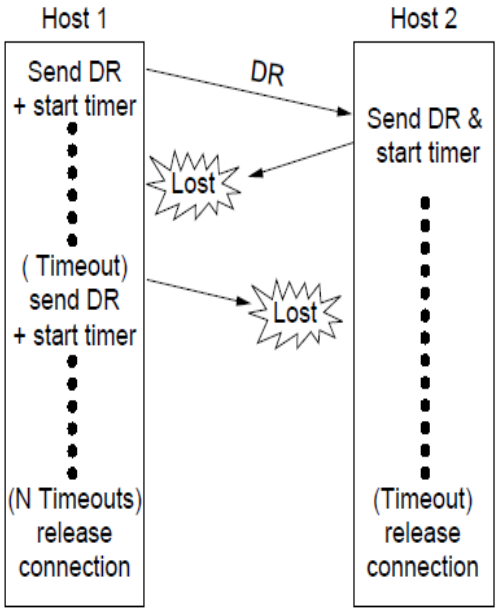
- Les cas exceptionnels sont traités à l'aide de minuterie et de retransmission




Dernier ACK perdu, Hôte 2 expire



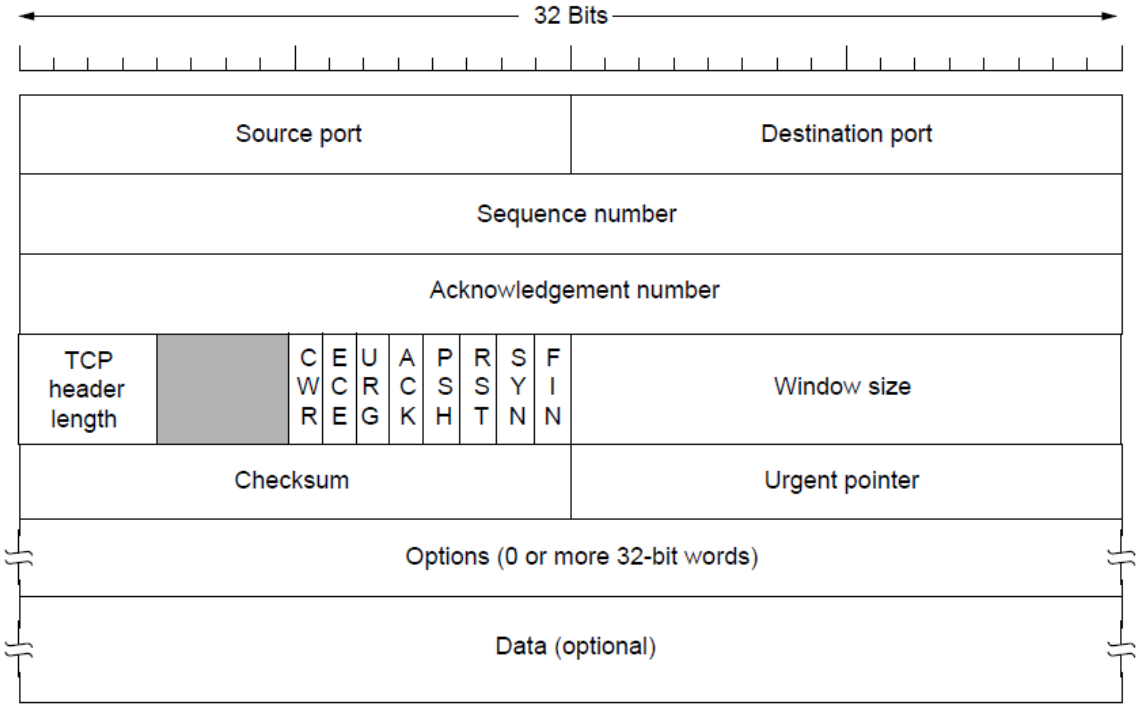

DR perdue entraîne retransmissions




Extrême : plusieurs DRs perdus entraînent les deux expirations

# Rappel : L'en-tête de segment TCP

- L'en-tête de segment TCP comprend des adresses (ports), une fenêtre dynamique (seq. /ACK.), **contrôle de flux (*window size*)**, contrôle d'erreur (*checksum*), etc.

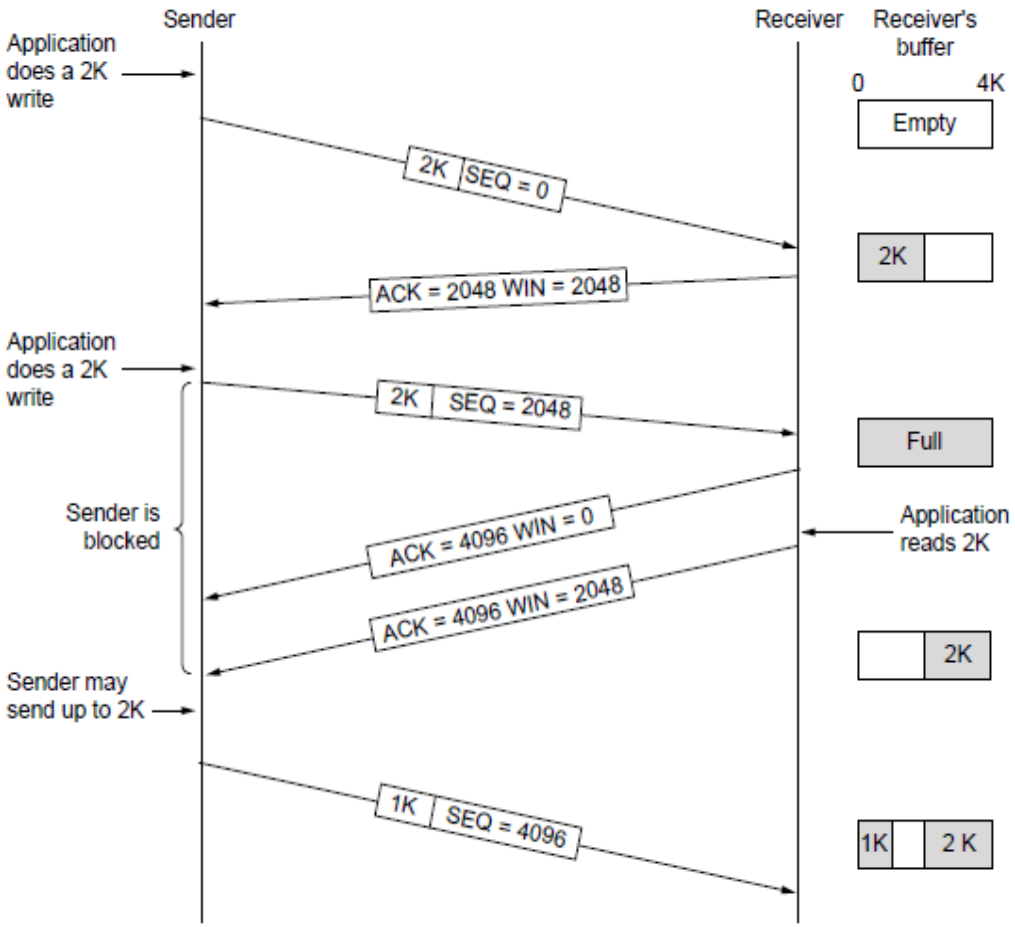


# Fenêtre dynamique TCP

- L'établissement d'une connexion TCP utilise des communications synchrones.
  - C'est-à-dire que l'expéditeur attend un accusé-réception (ACK) avant de continuer.
- Dans les exemples ci-haut, nous remarquons que le transfert de données est bidirectionnel et asynchrone une fois la connexion établie.
  - C'est-à-dire que le prochain segment de données peut être envoyé sans attendre un ACK des segments précédents.
- Afin de limiter la taille de donnée qu'il est prêt à recevoir sans ACK, chaque hôte annonce sa fenêtre dynamique (WS pour *Window Size*) dans les paquets SYN lors de la connexion
  - Cette valeur tampon peut changer en tout temps durant la connexion

# Fenêtre dynamique TCP

- TCP ajoute le contrôle de flux à la fenêtre dynamique
  - ACK + WIN est la limite du transmetteur.



# Accusés réception en retard

- Considérez Telnet
  - Paquet au récepteur avec un octet (41 octets)
  - ACK au transmetteur (40 octets)
  - Application réceptrice utilise l'octet et met à jour la fenêtre (40 octets)
  - Application réceptrice fait écho du caractère (41 octets)
  - Expéditeur ACK l'écho (40 octets)
  - Application expéditrice utilise l'octet et met à jour la fenêtre (40 octets)

- On doit être plus efficace!!
  - par exemple, attendre 500 msec pour le ACK
  - L'information de contrôle de flux peut voyager sur les paquets de données,
  - mais…

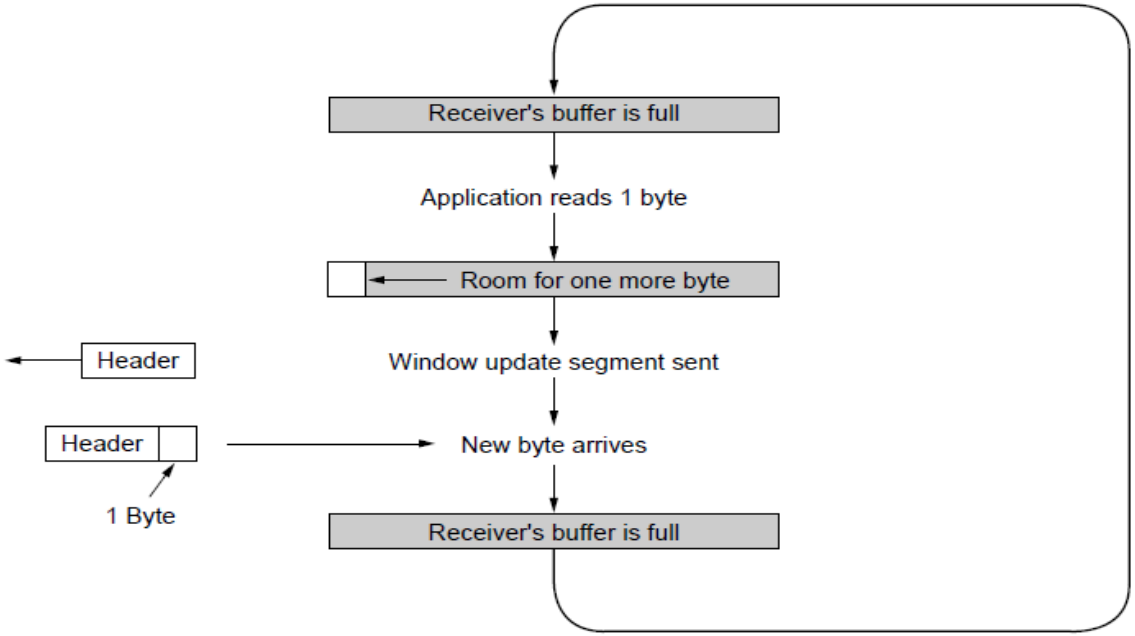
# Algorithme de Nagle

- Lorsque l'application expéditrice envoie des données en petits morceaux :
  - Envoie le premier morceau
  - Garde le reste des données dans un tampon en attendant la réception du premier ACK
  - Envoyer le restant du tampon
  - Tamponne les données de nouveau
  - Envoye le tampon lorsqu'on reçoit un ACK pour le segment suivant

  - On envoie également les données si le tampon atteint sa limite.

# Syndrome de la fenêtre idiote

- On doit traiter des cas spéciaux pour éviter les comportements indésirables
  - Syndrome de la fenêtre idiote au bas




Application du récepteur — lit un seul octet, le transmetteur envoie donc toujours un segment d'un octet

# La solution de Clark

- Quand le récepteur consomme les données en petits morceaux
  - On empêche le récepteur d'envoyer une mise à jour de fenêtre de 1 octet
  - On attend qu'on montant « raisonnable » de données, soit consommé, du tampon récepteur
    - On n'envoie une mise à jour de fenêtre que lorsque la plus petite des deux conditions suivantes arrive :
      - on attend la « grandeur de segment maximale » annoncée lors de la connexion (une option TCP)
      - le tampon est à moitié vide

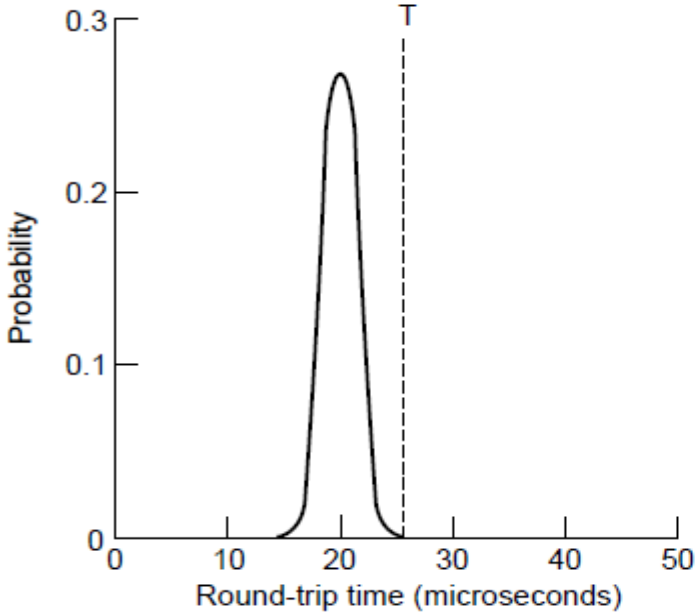
# Exemple de contrôle de flux

- Les données de A sont limitées par B

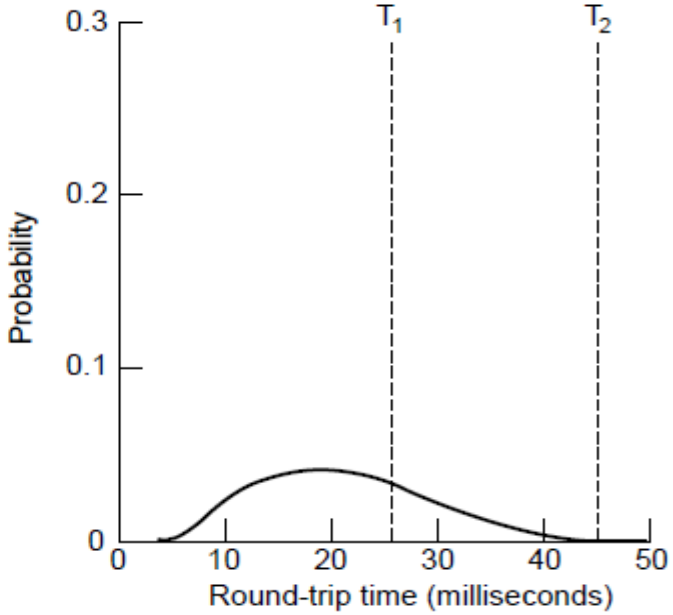
| | A | Message | B | B's Buffer | Comments |
|---|---|---|---|---|---|
| 1 | → | < request 8 buffers> | → | | A wants 8 buffers |
| 2 | ← | <ack = 15, buf = 4> | ← | 0 1 2 3 | B grants messages 0-3 only |
| 3 | → | <seq = 0, data = m0> | → | 0 1 2 3 | A has 3 buffers left now |
| 4 | → | <seq = 1, data = m1> | → | 0 1 2 3 | A has 2 buffers left now |
| 5 | → | <seq = 2, data = m2> | • • • | 0 1 2 3 | Message lost but A thinks it has 1 left |
| 6 | ← | <ack = 1, buf = 3> | ← | 1 2 3 4 | B acknowledges 0 and 1, permits 2-4 |
| 7 | → | <seq = 3, data = m3> | → | 1 2 3 4 | A has 1 buffer left |
| 8 | → | <seq = 4, data = m4> | → | 1 2 3 4 | A has 0 buffers left, and must stop |
| 9 | → | <seq = 2, data = m2> | → | 1 2 3 4 | A times out and retransmits |
| 10 | ← | <ack = 4, buf = 0> | ← | 1 2 3 4 | Everything acknowledged, but A still blocked |
| 11 | ← | <ack = 4, buf = 1> | ← | 2 3 4 5 | A may now send 5 |
| 12 | ← | <ack = 4, buf = 2> | ← | 3 4 5 6 | B found a new buffer somewhere |
| 13 | → | <seq = 5, data = m5> | → | 3 4 5 6 | A has 1 buffer left |
| 14 | → | <seq = 6, data = m6> | → | 3 4 5 6 | A is now blocked again |
| 15 | ← | <ack = 6, buf = 0> | ← | 3 4 5 6 | A is still blocked |
| 16 | • • • | <ack = 6, buf = 4> | ← | 7 8 9 10 | Potential deadlock |

# Gestion de minuteries TCP

- TCP estime la minuterie de retransmission des segments RTT (*round trip time*)
  - Note la moyenne et la variance (cas Internet)
  - La minuterie est fixée à $\mu + 4\sigma^2$




cas LAN :
RTT petits et réguliers




Cas Internet :
RTT plus long et irréguliers

# Comment choisir la minuterie

- Lorsque le réseau est chargé
  - Les erreurs/délais causent retransmissions
  - Les retransmissions entraînent une plus grande charge!

- On choisit la minuterie de façon dynamique
  - On s'adapte aux conditions réseau

- On calcule le SRTT (*Smoother Round-Trip Time*)
    - SRTT = α SRTT + (1 – α) R
    - R – Temps d'accusé réception courant
    - α – habituellement 7/8

- C'est une moyenne pondérée exponentielle EWMA (*Exponentially weighted moving average*)

# Comment choisir la minuterie

- La variance/sautillement (*jitter*) peut aussi être un problème
  - On améliore la technique en utilisant le RTTVAR (*Rount-Trip Time Variance*)
    - RRTVAR = β RTTVAR + (1 – β) |SRTT – R|
    - R – Temps d'accusé réception courant
    - β – habituellement 3/4

- Alors : RTO (*Retransmission Timeout*)

  - RTO = SRTT + 4 x RTTVAR
  (minimum 1s)

# Quelles valeur de SRTT utiliser ?

- Voici ce que dit la RFC 2988

https://tools.ietf.org/html/rfc2988

(2.3) When a subsequent RTT measurement R' is made, a host MUST set

RTTVAR <- (1 - beta) * RTTVAR + beta * |SRTT - R'|
SRTT <- (1 - alpha) * SRTT + alpha * R'

The value of SRTT used in the update to RTTVAR is its value before updating SRTT itself using the second assignment. That is, updating RTTVAR and SRTT MUST be computed in the above order.

# Example

- Supposons que le temps d'aller-retour TCP (RTT) est actuellement de 30,0 ms et que la variation RTT (RTTVar) est actuellement de 2,0 ms et que les accusés de réception arrivent après 26,0 et 24,0 ms.

- Quelle est la nouvelle estimation RTO utilisant l'algorithme Jacobson (en ms)?

- Utilisez $\alpha = 0{,}9$ et $\beta = 0{,}75$.

# Solution

alpha = 0.9 Beta = 0.75

| t | RTT | SRTT | RTTVar | RTO |
|---|---|---|---|---|
| 0 | | 30.0 | 2.0 | 38.0 |
| 1 | 26.0 | 29.6 | 2.4 | 39.2 |
| 2 | 24.0 | 29.0 | 3.05 | 41.2 |

Prochaine séance :

# EFFICACITÉ TCP : CONTRÔLE DE CONGESTION